

Advanced Data Management

Description Logic Ontologies

Domenico Fabio Savo

*Corso di laurea magistrale
INGEGNERIA INFORMATICA
Dipartimento di Ingegneria Gestionale, dell'Informazione e della Produzione
University of Bergamo*

Outline of the course

1. Introduction to propositional logic
 2. Introduction to First-order Logic
 3. Relational Calculus
 4. Information Integration Systems (IIS)
 5. Logical formalization of IISs
 6. Mapping between **Global Schema** e **Data Sources**
 7. Incomplete information databases
 8. Query answering over IISs
 9. Ontology-Based Data Management (OBDM)
 - 10. Description Logic Ontologies**
 11. Query answering in ontologies
 12. Query answering in OBDM
-

Ontologies for information management

Ontologies

Definition [Ontologies in Computer Science]: An **ontology** is a representation scheme describing a **formal conceptualization** of a domain of interest.

The specification of an **ontology** usually comprises two distinct levels:

- **Intensional level:** specifies a set of **conceptual elements** and of constraints/axioms describing the conceptual structures of the domain.
 - **Extensional level:** specifies a set of **instances** of the conceptual elements described at the intensional level.
-

Description Logic Ontologies

The formal foundations for ontology languages are in logic, and specifically in **Description Logics (DLs)**.

Description logics are fragments of **first-order logic** specifically tailored towards the representation of structured knowledge.

- Formalisms in logic are used to provide data with a **formal semantics**
- The logic-based formalization allows one to provide **automated support** for tasks related to data management, by means of **logic-based inference**.
- **Computational aspects** are of concern, so that tools can provide effective support for automated reasoning.

In this course we adopt DLs for expressing ontologies

Conceptual Schemas in Information Systems

Intensional information has traditionally played an important role in information systems.

Design phase of the information system:

- **Conceptual modeling**: from the requirements, a **conceptual schema** of the domain of interest is produced.
 - The **conceptual schema** is used to produce the **logical data schema**.
 - The data are stored according to the logical schema, and queried through it.

Conceptual Modeling: The activity of formally describing some aspects of the physical and social world around us for the purposes of understanding and communication.

Ontologies in Information Systems

The role of ontologies in information systems goes beyond that of conceptual schemas.

Ontologies affect the whole life-cycle of the information system:

- Ontologies, with the associated reasoning capabilities and inference tools, can provide support at design time.
- The use of ontologies can significantly simplify **maintenance** of the information system data assets.
- The ontology is used also to support the interaction with the information system and to improve communication between organizations
- Ontology-based Data Management

Issues in ontology-based information management

Which is the right ontology language?

- many proposals with different expressive power and complexity of inference have been made

Which languages should we use for querying?

- requirements for querying are different from those for modeling

How do we connect the ontology to available data sources?

- mismatch between the information in an ontology and data in a data source

How can we handle large ontologies?

- one needs to consider the **tradeoff** between expressive power and complexity of inference.

How can we cope with large amounts of data?

- what is good for large ontologies, may not be good enough for large amounts of data.

Introduction to Description Logics

What are Description Logics?

Description Logics (DLs) are logics specifically designed to represent and reason on structured knowledge.

The domain of interest is composed of objects and is structured into:

- **concepts**, which correspond to classes, and denote sets of objects
- **roles**, which correspond to (binary) relationships, and denote binary relations on objects

The knowledge is asserted through so-called **assertions**, i.e., logical axioms.

What are Description Logics about?

Abstractly, DLs allow one to predicate about labeled directed graphs:

- Nodes represent real world objects.
- Node labels represent types/categories of objects.
- Edges represent relations between (pairs of) objects, or between objects and values
- Edge labels represent the types of relations between objects, or between objects and values.

Every fragment of the world that can be abstractly represented in terms of a labeled directed graph is a good candidate for being represented by DLs.

What are Description Logics about?

A **DL** is characterized by:

A **description language**: how to form concepts and roles

- $\text{Human} \sqcap \text{Male} \sqcap \exists \text{hasChild} \sqcap \forall \text{hasChild}.(\text{Doctor} \sqcup \text{Lawyer})$

A mechanism to **specify knowledge** about concepts and roles (i.e., a **TBox**)

- $T = \{ \text{Father} \equiv \text{Human} \sqcap \text{Male} \sqcap \exists \text{hasChild};$
 $\text{HappyFather} \sqsubseteq \text{Father} \sqcap \forall \text{hasChild}.(\text{Doctor} \sqcup \text{Lawyer}) \}$

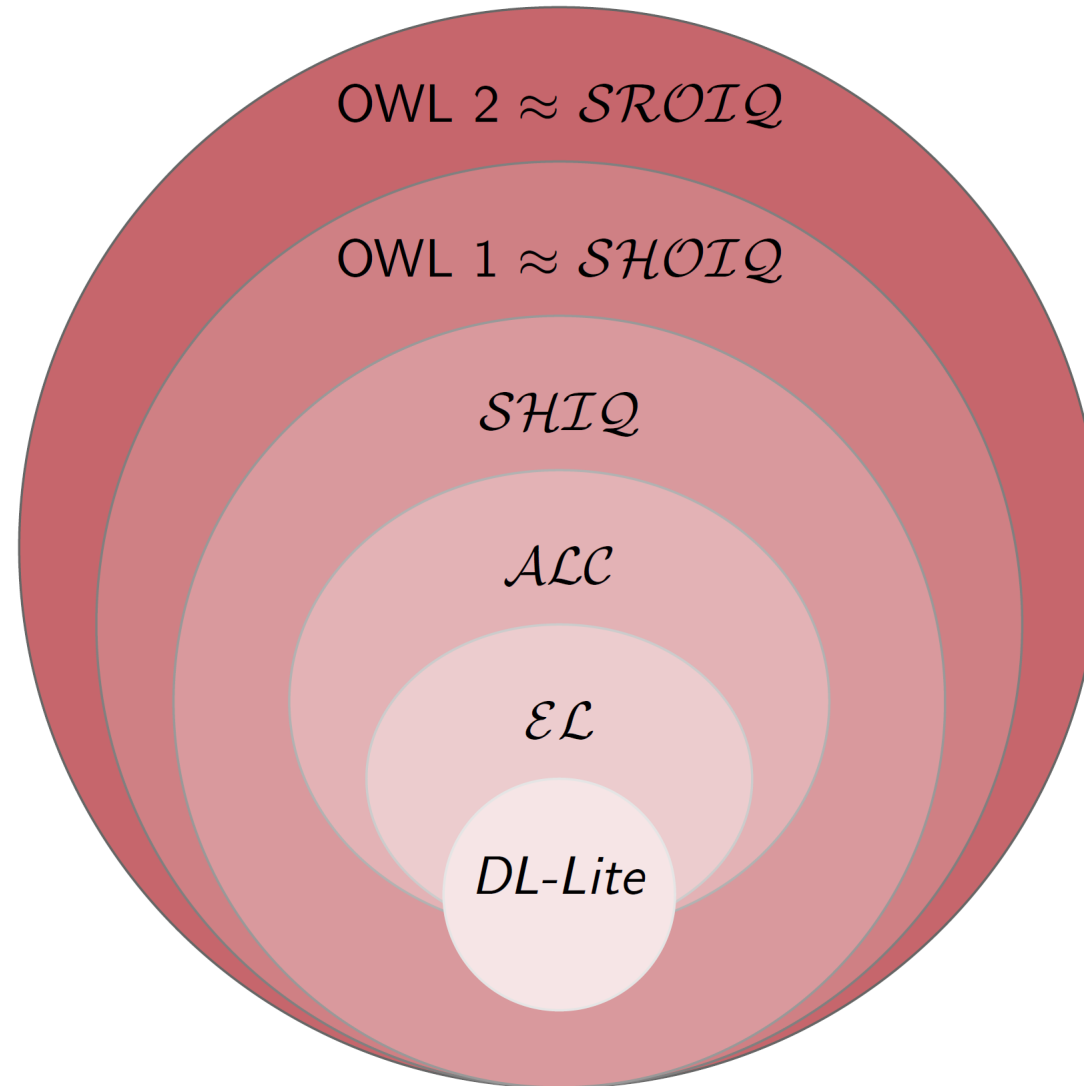
A mechanism to specify **properties of objects** (i.e., an **ABox**)

- $A = \{ \text{HappyFather}(\text{john}); \text{hasChild}(\text{john}, \text{mary}) \}$

A set of **inference services**: how to reason on a given ontology

- $T \cup A \models (\text{Doctor} \sqcup \text{Lawyer})(\text{mary})$

Many description logics



Description Language

A **description language** provides the means for defining:

- **concepts**, corresponding to classes: interpreted as sets of objects;
- **roles**, corresponding to relationships: interpreted as binary relations on objects.

To define concepts and roles:

- We start from a (countably infinite) alphabet of concept names and role names, forming the so-called **atomic concepts and roles**.
- Then, by applying specific **constructors**, we can build **complex concepts and roles**, starting from the atomic ones.

A **description language** is characterized by the set of **constructs** that are available for that.

Formal semantics of a description language

The **formal semantics** of DLs is given in terms of **interpretations**.

Definition: An **interpretation** $I = (\Delta^I, \cdot^I)$ consists of:

- a non-empty set Δ^I called the **interpretation domain** (of I)
- an **interpretation function** $\cdot^I$, which maps:
 - each atomic concept A to a subset A^I of Δ^I
 - each atomic role P to a subset P^I of $\Delta^I \times \Delta^I$

The **interpretation** function is extended to complex concepts and roles according to their syntactic structure.

Concept constructors

Construct	Syntax	Example	Semantics
Atomic concept	A	Student	$A^I \subseteq \Delta^I$
Atomic role	P	LivesIn	$P^I \subseteq \Delta^I \times \Delta^I$
Atomic negation	$\neg A$	$\neg$ Student	$\Delta^I \setminus A^I$
Conjunction	$C \sqcap D$	Student $\sqcap$ Worker	$C^I \cap D^I$
(unqual) existential restriction	$\exists R$	$\exists$ LivesIn	$\{ o \mid \exists o'. (o, o') \in R^I \}$
Value restriction	$\forall R. C$	$\forall$ LivesIn.City	$\{ o \mid \forall o'. (o, o') \in R^I \rightarrow o' \in C^I \}$
Bottom	$\perp$		$\emptyset$

(C , D denote arbitrary concepts and R an arbitrary role)

The above constructs form the basic language $\mathcal{AL}$ of the family of $\mathcal{AL}$ languages.

Additional concept and role constructors

Construct	Syntax	$\mathcal{AL}$	Semantics
Disjunction	$C \sqcup D$	$\mathcal{U}$	$C^I \cup D^I \subseteq \Delta^I$
Top	$\top$		Δ^I
(full) negation	$\neg C$	$\mathcal{C}$	$\Delta^I \setminus C^I$
Qual. existential restriction	$\exists R. C$	$\mathcal{E}$	$\{ o \mid \exists o'. (o, o') \in R^I \wedge o' \in C^I \}$
Number restriction	$(\geq k R)$	$\mathcal{N}$	$\{ o \mid \#\{o' \mid (o, o') \in R^I\} \geq k \}$
	$(\leq k R)$		$\{ o \mid \#\{o' \mid (o, o') \in R^I\} \leq k \}$
Inverse role	R^-	$\mathcal{I}$	$\{(o', o) \mid (o, o') \in R^I\}$
$\vdots$	$\vdots$	$\vdots$	$\vdots$

Many different DL constructs and their combinations have been investigated.

DL constructs – Example

- Disjunction $\forall hasChild. (Doctor \sqcup Lawyer)$
- Inverse role $\exists hasChild^{-}$
- (full) negation $\neg(Doctor \sqcup Lawyer)$
- Qualified existential restriction $\exists hasChild^{-}. Doctor$
- Number restriction $(\geq 2 \text{ hasChild}) \sqcap (\leq 1 \text{ sibling})$

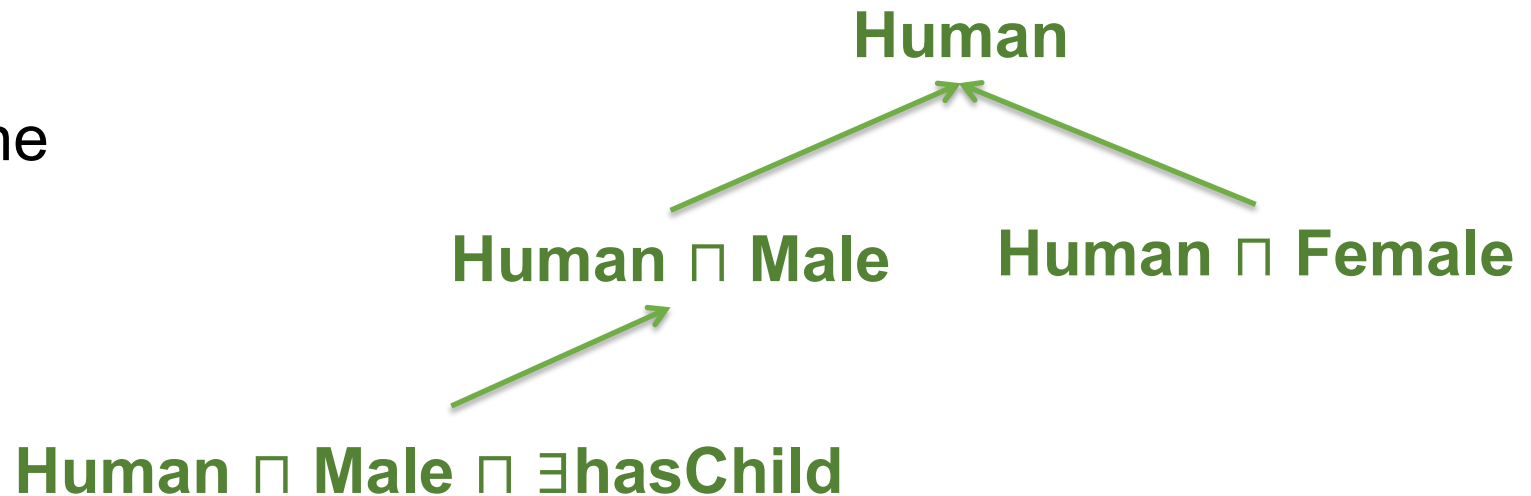
Reasoning over concept expressions

An **interpretation** I is a model of a concept C if $C^I \neq \emptyset$.

Basic reasoning tasks:

- Concept **satisfiability**: does C admit a model?
- Concept **subsumption**: $C \sqsubseteq D$: is $C^I \subseteq D^I$ for every interpretation I ?

Subsumption is used to build the **concept hierarchy**



Complexity of reasoning over concept expressions

Complexity of concept satisfiability:

$\mathcal{AL}, \mathcal{ALN}$	PTime
$\mathcal{ALU}, \mathcal{ALUN}$	NP-Complete
$\mathcal{ALE}$	CoNP-Complete
$\mathcal{ALC}, \mathcal{ALCI}, \mathcal{ALCN}$	PSpace-Complete

Observations: two sources of complexity:

- union ($\mathcal{U}$) of type **NP**,
- qualified existential restriction ($\mathcal{E}$) of type **coNP**.

With more combinations the complexity jumps to **PSpace**.

Number restrictions ($\mathcal{N}$) do not add to the complexity.

Structural properties vs. asserted properties

So far, we have seen how to build **complex concept and roles expressions**, which allow one to denote classes with a complex structure.

However, in order to represent real world domains, one needs the ability to **assert properties** of classes and relationships between them (e.g., as done in ER and UML class diagrams).

The assertion of properties is done in DLs by means of an **ontology** (or knowledge base).

Description Logics ontology (or knowledge base)

A DL ontology is a pair $\mathbf{O} = \langle \mathbf{T}, \mathbf{A} \rangle$, where $\mathbf{T}$ is a **TBox** and $\mathbf{A}$ is an **ABox**

Definition: A DL **TBox** consists of a set of assertions on concepts and roles:

- Inclusion assertions on concepts: $C_1 \sqsubseteq C_2$
- Inclusion assertions on roles: $R_1 \sqsubseteq R_2$
- Property assertions on (atomic) roles such as:

(**transitive P**), (**symmetric P**), (**functional P**), (**reflexive P**) ...

Definition: A DL **ABox** consists of a set of assertions on **individuals** (we use c_i to denote individuals):

- Membership assertions for concepts: $D(c)$
- Membership assertions for roles: $R(c_1, c_2)$
- Equality and distinctness assertions: $c_1 \approx c_2$; $c_1 \not\approx c_2$

Description Logics ontology – Example

TBox assertions:

Concept inclusion assertions:

HappyFather $\sqsubseteq \forall \text{descendant} . (\text{Doctor} \sqcup \text{Lawyer} \sqcup \text{HappyPerson})$

HappyAnc $\sqsubseteq \forall \text{descendant} . \text{HappyFather}$

Teacher $\sqsubseteq \neg \text{Doctor} \sqcap \neg \text{Lawyer}$

Role inclusion assertions:

hasChild $\sqsubseteq \text{descendant}$

hasFather $\sqsubseteq \text{hasChild}^{-}$

Property assertions on roles:

(transitive descendant), (reflexive descendant), (functional hasFather)

ABox assertions:

Teacher(mary), hasFather(mary, john), HappyAnc(john)

Semantics of a DL ontology

The **semantics** is given by specifying when an interpretation I satisfies an assertion α , denoted by $I \models \alpha$

- **TBox** assertions:

$$I \models \mathbf{C}_1 \sqsubseteq \mathbf{C}_2 \text{ if } C_1^I \subseteq C_2^I$$

$$I \models \mathbf{R}_1 \sqsubseteq \mathbf{R}_2 \text{ if } R_1^I \subseteq R_2^I$$

$$I \models (\mathbf{prop } P) \text{ if } P^I \text{ is a relation that has property } \mathbf{prop}$$

- **ABox** assertions:

We need first to extend the interpretation function I , so that it maps each individual $\mathbf{c}$ to an element c^I of Δ^I .

$$I \models \mathbf{A}(\mathbf{c}) \text{ if } c^I \in A^I$$

$$I \models \mathbf{P}(\mathbf{c}_1, \mathbf{c}_2) \text{ if } (c_1^I, c_2^I) \in P^I$$

$$I \models \mathbf{c}_1 \approx \mathbf{c}_2 \text{ if } c_1^I = c_2^I$$

$$I \models \mathbf{c}_1 \not\approx \mathbf{c}_2 \text{ if } c_1^I \neq c_2^I$$

Model of a Description Logics ontology

Definition: An interpretation I is a **model** of:

- An assertion α , if it satisfies α , i.e., $I \models \alpha$
- A **TBox** T , if it satisfies all the assertions in T
- An **ABox** A , if it satisfies all the assertions in A
- An ontology $O = \langle T, A \rangle$, if it is a model for both T and A .

In what follows, we will use $I \models \beta$ to denote the fact that I is a model for β (where β stands for an assertion, **TBox**, **ABox**, or **ontology**). Moreover, we will use **Mod**(O) to denote the set of models of the ontology O .

Reasoning in Description Logics

Logical implication

The fundamental reasoning service from which all other ones can be easily derived is **logical implication**.

Definition (Logical implication): An ontology $\mathbf{O}$ logically implies an assertion α , written $\mathbf{O} \models \alpha$, if α is satisfied by all models of $\mathbf{O}$, i.e., $I \models \alpha, \forall I \in \text{Mod}(\mathbf{O})$.

We can provide an analogous definition for a **TBox T** instead of an ontology $\mathbf{O}$.

TBox reasoning

TBox Satisfiability: T is satisfiable, if it admits at least one model.

Concept Satisfiability: C is satisfiable w.r.t. T , if there is a model I of T such that C^I is not empty, i.e., $T \not\models C \equiv \perp$

Subsumption: C_1 is subsumed by C_2 w.r.t. T , if for every model I of T we have $C_1^I \subseteq C_2^I$, i.e., $T \models C_1 \sqsubseteq C_2$

Equivalence: C_1 and C_2 are equivalent w.r.t. T if for every model I of T we have $C_1^I = C_2^I$, i.e., $T \models C_1 \equiv C_2$

Disjointness: C_1 and C_2 are disjoint w.r.t. T if for every model I of T we have $C_1^I \cap C_2^I = \emptyset$, i.e., $T \models C_1 \sqcap C_2 \equiv \perp$

Functionality implication: A functionality assertion (*funct* R) is logically implied by T if for every model I of T , we have that $(o, o_1) \in R^I$ and $(o, o_2) \in R^I$ implies $o_1 = o_2$, i.e., $T \models (\text{funct } R)$.

Analogous definitions hold for role satisfiability, subsumption, equivalence, and disjointness.

Reasoning over an ontology

Ontology Satisfiability: Verify whether an ontology O is satisfiable, i.e., whether O admits at least one model.

Concept Instance Checking: Verify whether an individual c is an instance of a concept C in every model of O , i.e., whether $O \models C(c)$.

Role Instance Checking: Verify whether a pair (c_1, c_2) of individuals is an instance of a role R in every model of O , i.e., whether $O \models R(c_1, c_2)$.

Query Answering: see later . . .

Reasoning in Description Logics – Exercise

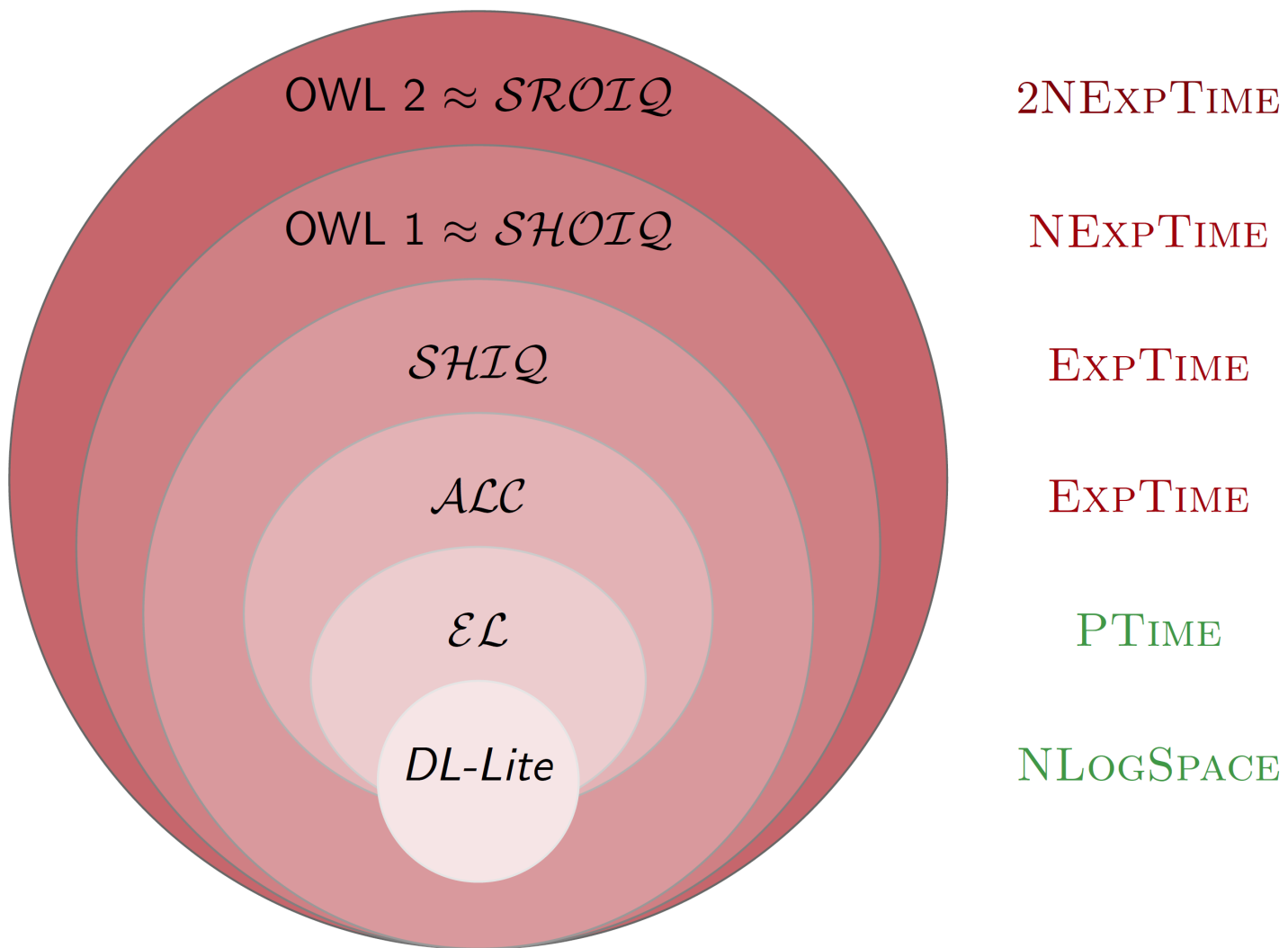
Consider the ontology $\mathbf{O} = \langle \mathbf{T}, \mathbf{A} \rangle$, where:

$\mathbf{TBox\ T} = \{ \text{HappyFather} \sqsubseteq \forall \text{descendant.}(\text{Doctor} \sqcup \text{Lawyer} \sqcup \text{HappyPerson}),$
 $\text{HappyAnc} \sqsubseteq \forall \text{descendant.HappyFather},$
 $\text{Teacher} \sqsubseteq \neg \text{Doctor} \sqcap \neg \text{Lawyer},$
 $\text{hasChild} \sqsubseteq \text{descendant},$
 $\text{hasFather} \sqsubseteq \text{hasChild}^{-},$
 (transitive descendant),
 (reflexive descendant),
 (functional hasFather) }

$\mathbf{ABox\ A} = \{ \text{Teacher(mary)}, \text{hasFather(mary, john)}, \text{HappyAnc(john)} \}$

We have that: $\mathbf{O} \models \text{HappyPerson(mary)}$. Why?

Complexity of reasoning over DL ontologies



Reasoning over DL ontologies is a **complex task**:

Bad news:

- without restrictions on the form of TBox assertions, reasoning over DL ontologies is already **ExpTime-hard**, even for very simple DLs

Good news:

- There are DL reasoners that perform reasonably well in practice for such DLs (e.g, Racer, Pellet, Fact++, . . .)

DLs vs other Conceptual Modeling Languages

Most DLs are **well-behaved** fragments of **First Order Logic (FOL)** – differently from FOL theories, reasoning over a DL ontology is **decidable**.

It is possible to encode DL ontologies in FOL theories and reduce DL reasoning services to FOL reasoning services.

DLs are nowadays considered the foundations for ontology languages. **OWL (Web Ontology Language)**, the W3C standard for specifying ontology in the *Semantic Web*, has been defined as syntactic variants of certain DLs.

DLs are also ideally suited to capture the fundamental features of conceptual modeling formalism used in information systems design such as:

- **Entity-Relationship diagrams**, used in database conceptual modeling
- **UML Class Diagrams**, used in the design phase of software applications

Credits

The presentations of this module of the course are based on the original slides of Prof. Giuseppe De Giacomo, Prof. Diego Calvanese, Prof. Marco Console, Prof. Maurizio Lenzerini, and Prof. Domenico Lembo.
